Race Conditions

Although threading is a very useful concept it has several challenges associated with it

© COPYRIGHT INTELLIPAAT.COM. ALL RIGHTS RESERVED.

Race Conditions

One of these several challenges is called race conditions

© COPYRIGHT INTELLIPAAT.COM. ALL RIGHTS RESERVED.

Race Conditions

A Race Condition is a situation where we get incorrect results because multiple threads are manipulating the same data or resource

© COPYRIGHT INTELLIPAAT.COM. ALL RIGHTS RESERVED.

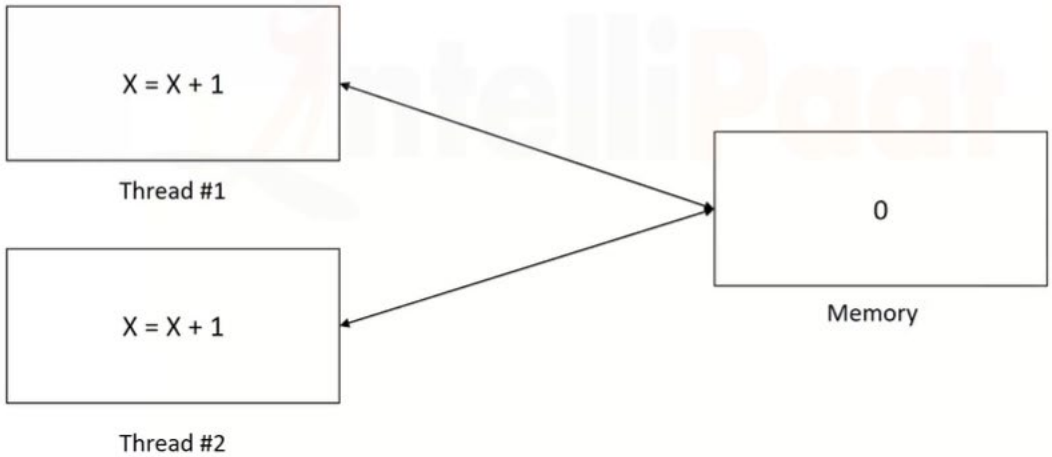
Race Conditions

Since these multiple threads are first accessing and then manipulating the data all at once, it is possible the end up manipulating some old data while another thread has already updated the data in memory

© COPYRIGHT INTELLIPAAT.COM. ALL RIGHTS RESERVED.

Race Conditions

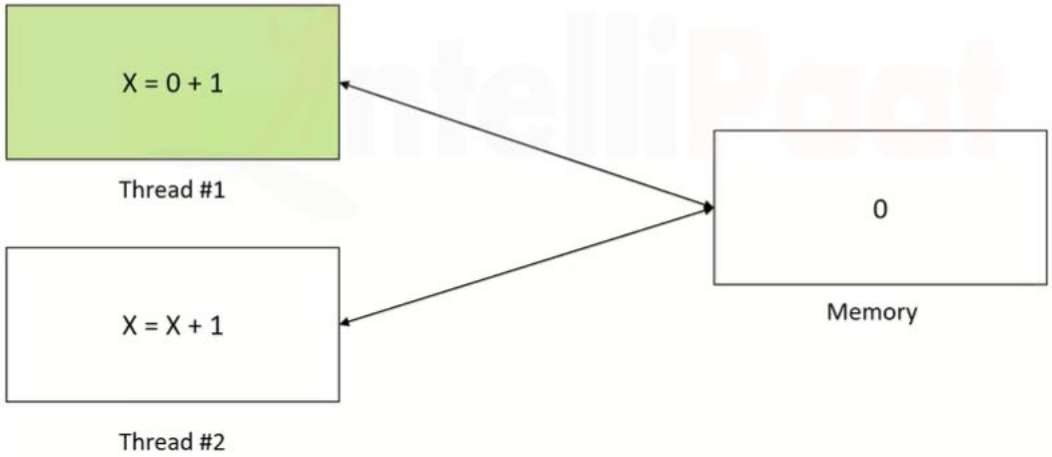
For example, consider two threads trying to increment the value of a counter variable which is initially set to zero



© COPYRIGHT INTELLIPAAT.COM. ALL RIGHTS RESERVED.

Race Conditions

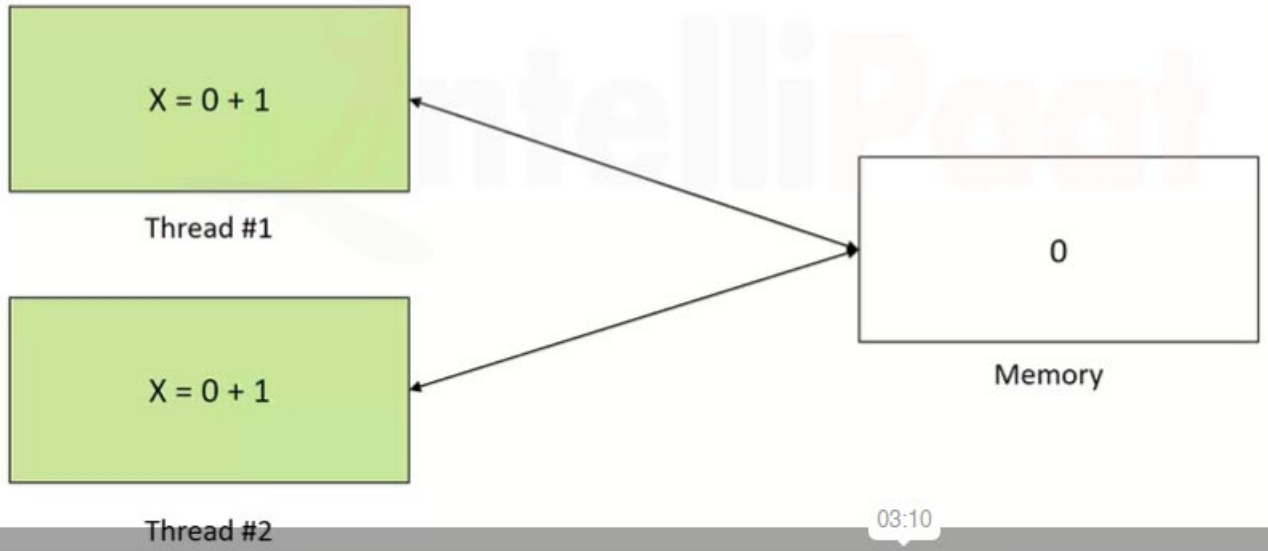
First thread accesses the value (0) and increments it to 1 and then puts it the memory



© COPYRIGHT INTELLIPAAT.COM. ALL RIGHTS RESERVED.

Race Conditions

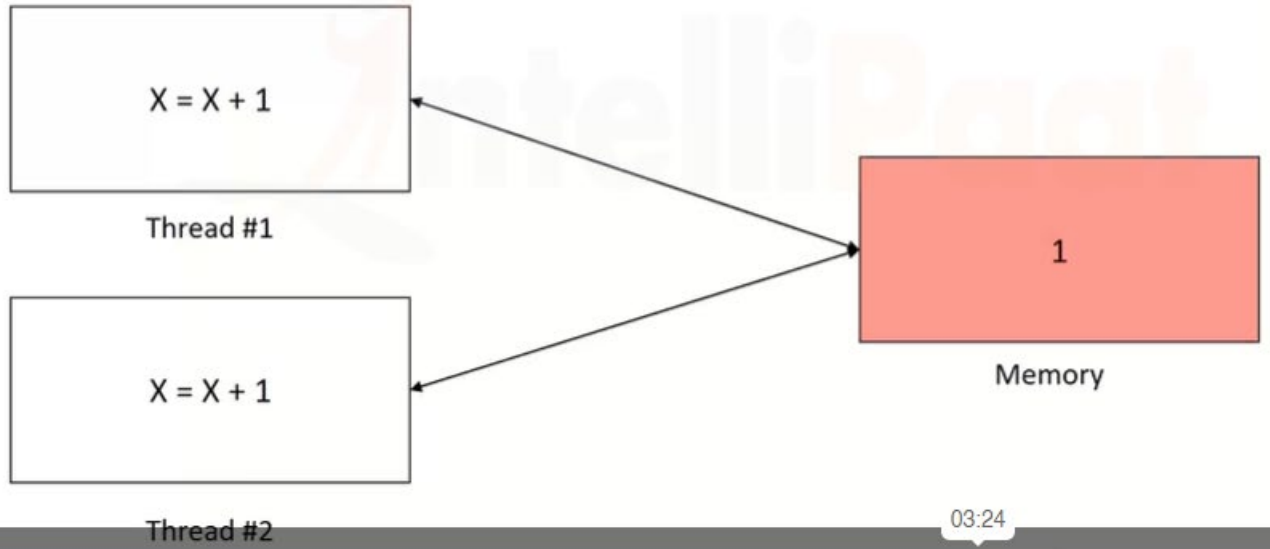
In the same time second thread also accessed the value (0) and increments it to 1 and then puts it the memory



© COPYRIGHT INTELLIPAAT.COM. ALL RIGHTS RESERVED.
WWW.INTELLIPAAT.COM

Race Conditions

So now the value instead of being two is 1 because both the threads did not wait for each other to finish



© COPYRIGHT INTELLIPAAT.COM. ALL RIGHTS RESERVED.
WWW.INTELLIPAAT.COM

Synchronization using Locks

One of the easiest way in Python is make use of Locks

© COPYRIGHT INTELLIPAAT.COM. ALL RIGHTS RESERVED.

Synchronization using Locks

Locks allow us to avoid race conditions allowing only one thread at a time into the read-modify-write section of your code

© COPYRIGHT INTELLIPAAT.COM. ALL RIGHTS RESERVED.

Synchronization using Locks

A Lock is an object that acts like a pass or a token which allows only one thread at a time to have access to it

© COPYRIGHT INTELLIPAAT.COM. ALL RIGHTS RESERVED.

Synchronization using Locks

Any other thread that wants the access to the same Lock must wait until the current owner of the Lock gives it up

© COPYRIGHT INTELLIPAAT.COM. ALL RIGHTS RESERVED.

Synchronization using Locks

For locks in python you would have to use Lock class form the threading package

© COPYRIGHT INTELLIPAAT.COM. ALL RIGHTS RESERVED.

Synchronization using Locks

We can manually create the lock object by simply instantiating an object Lock class

© COPYRIGHT INTELLIPAAT.COM. ALL RIGHTS RESERVED.

Synchronization using Locks

Then we can call the acquire method to get access to the lock and use the release method to give up the access to lock so other threads can acquire it

© COPYRIGHT INTELLIPAAT.COM. ALL RIGHTS RESERVED.

Synchronization using Locks

To do it more succinctly you can use a lock using the with keyword so that python can all the acquire and release methods on their own at the start and end of the block, respectively

© COPYRIGHT INTELLIPAAT.COM. ALL RIGHTS RESERVED.